

LOGICLENS

POWERED BY TRACEWISE AI

The Complete End-to-End System Manual, Architecture Specification & Developer Handover Blueprint

A self-contained production-grade repository blueprint and execution guide. This reference manual maps every architectural component, execution tracing parser, schema entity, and user interface state-machine in the LogicLens codebase. Designed to enable high-velocity development without prior chat history context.

SYSTEM DATA & SPECIFICATIONS

System Architect:	Antigravity AI Co-Pilot (DeepMind Engineering Group)
Project Type:	Full-Stack Educational Visualizer (SaaS Paradigm)
Development State:	Phase 2 Completed (Production Ready & Split Deployed)
Active Stack:	TypeScript Monorepo, React Flow, Zustand, Monaco, Express, Drizzle, Stripe
Repository URL:	https://github.com/mohdazhar2417-coder/Hackathon-Starter-Kit-M
Reference Date:	May 17, 2026
Target Audience:	Future Engineering Agents, Core Developers, Hackathon Jurors

HANDOVER ENGINE READY: SOURCE CODE PARSED & ARCHITECTED

DOCUMENT TABLE OF CONTENTS

Section 1: Executive Summary & Core Product Concept	Page 3
High-level business & educational values, problem-solution statements, and TraceWise AI alignment.	
Section 2: Architecture Blueprint & Technical Stack	Page 4
A complete mapping of the pnpm monorepo layout, libraries, package dependencies, and sharing protocols.	
Section 3: Database & Drizzle Schema Dictionary	Page 5
Structured schema breakdown of PostgreSQL tables, fields, constraints, relations, and billing integrations.	
Section 4: The TraceWise AI Tracing & Simulation Engine	Page 6
Inner mechanisms of parser, tokenizer, symbolic executor, graph layout generator, and step calculator.	
Section 5: The LogicLens UI Workspace & Workspace Panels	Page 8
The 3-column command layout, Monaco Editor sync, React Flow flowchart node engine, and memory visualizer.	
Section 6: Complete Curriculum & Sample Program Library	Page 10
The structured five-category sample program catalog, data models, and core demo-critical tracing mappings.	
Section 7: Production Deployment & Split Config Blueprint	Page 11
Vercel frontend, Render backend, Neon DB pipelines, environment variables, and CORS policy mappings.	
Section 8: Future Development Roadmap & Developer Handover	Page 12
The developer playbook: How to add features, extend engines, and execute Phases 3 to 6 technical backlogs.	

1. EXECUTIVE SUMMARY & CORE PRODUCT CONCEPT

1.1 The "Black Box" Problem in Coding Education

When beginner students start learning Java, they struggle to visualize what happens behind the scenes. Standard IDEs run programs in a millisecond and spit out terminal outputs. This creates a "black box" where variables are magically updated, control loops are memorized, and conditional branches are misunderstood. Students struggle to trace the call stack, visual memory representations (JVM stack and heap), and arithmetic operations (especially core behaviors like Java integer division and loop boundary mutations).

1.2 The LogicLens Framework

LogicLens bridges the cognitive gap between "writing code" and "understanding execution." It takes beginner-structured Java source code, parses it, executes it step-by-step in a sandboxed, state-driven virtual simulation engine (TraceWise AI), and renders a real-time visual playback. The execution is unrolled, making loop structures, condition jumps, output concatenation, and stack frames extremely explicit.

Core Tagline & Elevator Pitch

LogicLens: See how beginner Java programs actually think.
Instead of standard step-over debuggers, LogicLens decomposes every loop iteration, variable assignment, and division calculation into highly detailed, interactive, and beautifully unrolled flow models.

1.3 Target Audience and Real-World Value

- CS1 Students: Provides a visual model of assignments, loops, and pattern program execution.
- Bootcamp Self-learners: Replaces manual paper-tracing of algorithms with real-time memory state visualization.
- Educators: Serves as an interactive teaching whiteboard showing how variables evolve step-by-step.
- Interview Candidates: Clarifies fundamental algorithms like prime number, palindrome check, and reverse arithmetic.

2. ARCHITECTURE BLUEPRINT & TECHNICAL STACK

2.1 Monorepo Layout & System Structure

LogicLens is structured as a pnpm monorepo. This allows shared types, schemas, and specs to reside in reusable packages, while separation of concerns is maintained between the frontend rendering layer and the backend Express database orchestrator.

```
logiclens-workspace/
% % % frontend/                # React Single Page App (Vite Dev Engine)
%   % % % src/
%   %   % % engines/           # TraceWise AI Tracing Sub-systems
%   %   % % components/       # React Flow canvas, Monaco Editor wrapper, Panels
%   %   % % pages/            # Workspace, Landing, Auth, Dashboard, Admins
%   %   % % hooks/            # Zustand state management and queries
% % % backend/                 # Express API Server (Node/TypeScript)
%   % % % src/
%   %   % % controllers/      # Auth handlers, trace saving, billing controllers
%   %   % % index.ts          # Server initialization and routes
% % % packages/                # Shared local modules
%   % % % db/                 # Drizzle ORM schemas, pool configuration, migrations
%   % % % api-spec/           # Shared REST contracts
%   % % % api-zod/            # Zod validation models
% % % database/                # Schema definition and drizzle.config
```

2.2 Shared Package Dependency Grid

Dependency Name	Primary Use Area	Description
React Flow v11	Frontend Workspace	Drives the interactive flowchart canvas, active node glowing, and loops.
Monaco Editor React	Frontend Workspace	Fully-featured code editor with syntax highlighting and line indicators.
Zustand	Frontend Client State	Syncs Code Editor line highlights, Timeline Scrubber, and Memory Grid.
Framer Motion	Frontend UI	Drives the fluid Antigravity glassmorphic animations and transition states.
Drizzle ORM	Backend / Database	Provides type-safe SQL construction and schema management for PostgreSQL
Stripe SDK	Backend Billing	Orchestrates premium user subscription plans, customer profiles, and webhooks
JWT & Passport.js	Backend Auth	Implements local signup/login authentication and Google OAuth 2.0.

2.3 Core Package Operations

To manage the pnpm workspace efficiently, use the following core scripts:

- Install all dependencies: "pnpm install" at the root workspace.
- Run Dev Mode concurrently: "pnpm dev" launches frontend (5173) and backend API (5000) simultaneously.
- Apply migrations: "pnpm db:push" pushes schema directly using Drizzle.
- Build project: "pnpm build" compiles shared libraries, backend bundles, and frontend client assets.

3. DATABASE & DRIZZLE SCHEMA DICTIONARY

3.1 Database Topology

The persistence layer is powered by PostgreSQL (hosted in production on Neon DB, and simulated locally/in development using SQLite or memory tables). Drizzle ORM acts as the type-safe mapping layer, defining schemas as standard TypeScript files in the database module.

3.2 Schema Data Dictionary

1. Users Table (users)

Field Name	Type / Constraints	Default	Role / Details
id	serial (Primary Key)	nextval()	Unique internal user identifier.
name	text (Not Null)	none	User profile name.
email	text (Unique, Not Null)	none	Credentials email and billing key.
password_hash	text (Not Null)	none	Securely hashed password (bcrypt).
role	text (Not Null)	student	Role for authorization: "student" or "admin".
stripe_customer_id	text (Nullable)	null	Connected Stripe account reference.
subscription_status	text (Nullable)	none	Plan status: active, canceled, past_due, none.
plan_type	text (Nullable)	free	Limits execution tracing allowance: free, pro, enterprise.
created_at	timestamp with timezone	now()	Profile registration timestamp.

2. Saved Traces Table (saved_traces)

Field Name	Type / Constraints	Default	Role / Details
id	serial (Primary Key)	nextval()	Trace identifier.
user_id	integer (References users.id)	none	Foreign key identifying the trace owner.
title	text (Not Null)	none	Human readable label for the saved simulation.
category	text (Not Null)	none	Java curriculum group (e.g., Loops).
subtype	text (Not Null)	none	Algorithm indicator (e.g., Palindrome).
code	text (Not Null)	none	Raw Java code block input.
custom_inputs	jsonb (Nullable)	null	User custom inputs (e.g., {"n": 12321}).
trace_summary	text (Nullable)	null	AI-generated final calculation result summary.
final_output	text (Nullable)	null	Emulated standard output (console log stream).
saved_at	timestamp with timezone	now()	Trace persistence timestamp.
share_slug	text (Unique, Nullable)	null	Slug for public sharing of execution traces.
is_public	boolean (Not Null)	false	Flag for public access.

3. Favorite Programs Table (favorite_programs)

Field Name	Type / Constraints	Default	Role / Details
id	serial (Primary Key)	nextval()	Unique favorite tracker record.
user_id	integer (References users id)	none	Active user owner ID.
program_id	integer (Not Null)	none	Standard program library identifier.
program_name	text (Not Null)	none	Cached sample program name.
program_category	text (Not Null)	none	Cached category identifier.
added_at	timestamp with timezone	now()	Addition timestamp.

4. Sample Programs Catalog Table (sample_programs)

Field Name	Type / Constraints	Default	Role / Details
id	serial (Primary Key)	nextval()	Program library key.
name	text (Not Null)	none	Name (e.g., Celsius to Fahrenheit).
category	text (Not Null)	none	Educational category.
subtype	text (Not Null)	none	Curriculum subtype.
code	text (Not Null)	none	Pre-written beginner-level Java source.
description	text (Not Null)	none	Step-by-step description of the algorithm.
difficulty	text (Not Null)	beginner	Difficulty level: beginner, intermediate, advanced.
featured	boolean (Not Null)	false	Showcase indicator for the dashboard slider.
tags	text array (Not Null)	[]	Keywords for category searching.
created_at	timestamp with timezone	now()	Catalog generation date.

4. THE TRACEWISE AI TRACING & SIMULATION ENGINE

4.1 Tracing Pipeline Mechanics

The custom simulation engine (located at `frontend/src/engines/simulate/simulationEngine.ts`) parses, normalizes, executes, and builds visualizations for structured Java code entirely in the sandbox environments. This design avoids server execution latency or malicious infinite-loop threats. The engine works in 5 synchronous steps:

- Tokenization: Normalizes braces and semicolons; parses nested blocks, for headers, and while conditions.
- Parsing: Converts tokens into structured statements (declarations, assignments, print commands, conditionals, loops).
- Symbolic Execution: Steps through statements, updating a virtual environment and capturing memory states.
- Graph Generation: Builds a graph representing control flow with nodes and edges for React Flow.
- Explanation Compiling: Translates executing steps into educational text explanations and hints.

4.2 The Symbolic Execution Model

The engine tracks variables using a "VariableState" object, capturing before and after states. It records calculation breakdowns for assignments, e.g., `reversed * 10 + rem -> 0 * 10 + 1 = 1`.

```
export interface ExecutionStep {
  stepIndex: number;
  stepNumber: number;
  lineNumber: number;
  type: "assignment" | "loop" | "condition" | "output" | "start" | "end";
  title: string;
  codeLine: string;
  description: string;
  beforeState: VariableState;
  afterState: VariableState;
  stateChanges: { varName: string; before: any; after: any }[];
  calculations: { expr: string; result: any }[];
  iteration?: number;
  nodeId: string;
  loopId?: string;
  accumulatedOutput: string;
  branchTaken?: "true" | "false";
  status: "completed" | "current" | "pending";
}
```

4.3 Real-World Java Semantics Mapping

A major challenge in browser-based symbolic execution is replicating accurate Java semantics in a JavaScript environment. The LogicLens engine includes special workarounds for these edge cases:

Java Integer Division Emulation

In Java, `5 / 2` yields 2 (truncating decimals), whereas in JavaScript, `5 / 2` yields 2.5. The simulation engine resolves this by parsing the expression and automatically wrapping division operands in integer checks, truncating decimal values when executing integer-defined division.

Loop Iteration Unrolling

Unlike standard debuggers that pause on a loop header, LogicLens flattens execution loops into discrete, iteration-indexed steps. This allows students to scrub backwards and forwards through individual iterations.

4.4 Graph Construction and Layout

The engine dynamically generates React Flow elements using two algorithms:

1. buildFlowNodes.ts

Processes the flat syntax tree. Blocks are converted into nodes with visual labels: operation (rectangular), decision (diamond), loop (hexagonal), and input/output (parallelogram) nodes.

2. buildFlowEdges.ts

Connects nodes sequentially. Loops include loop-back edges pointing to condition checks, labeled with iteration indicators. If-else branches generate edges labeled with "true" and "false" markers.

Active Node Highlights

During execution, React Flow nodes are styled with neon green shadows representing execution status. Inactive paths are faded, visually demonstrating conditional jumps.

5. THE LOGICLENS UI WORKSPACE & WORKSPACE PANELS

5.1 The 3-Column Command Layout

The workspace is optimized for educational clarity, presenting code, flowcharts, and memory in a single view:

- Left (Code Editor): Monaco Editor wrapper. Synchronizes current execution steps, highlights lines, and auto-scrolls to keep active structures in focus.
- Center (Execution Canvas): React Flow canvas displaying a real-time trace of the program structure.
- Right (Memory Visualizer): Renders local stack frames, heap references, and variable updates. Updated variables flash with yellow neon accents.

5.2 Bottom Analysis Suite

Below the 3-column layout, four analytics panels offer additional educational insights:

1. Emulated Terminal Console

Displays character-by-character standard outputs, allowing students to trace how shapes are built line-by-line.

2. Loop Iteration Trace Table

A structured table displaying variable changes for loop indices across iterations, replacing manual trace tables.

3. Onboarding Tour Wizard

A guided tour using visual popups that highlights key controls like Step Forward, Step Back, and Custom Inputs.

4. Final Validation Banner

Triggers at the end of execution, verifying the final outputs of algorithm checks (e.g., "Prime Verified").

5.3 Admin Command Center

Admins can access a dashboard with metrics, usage overviews, featured program managers, and system control suites.

Admin Component	Access Level	Core Capabilities
Activity Log Tracker	Admin Only	Tracks user trace counts, custom program runs, and logins.
Curriculum Manager	Admin Only	Allows admins to edit, update, delete, or add sample programs.
Featured Switch	Admin Only	Pins successful programs to the dashboard carousel.
Billing Metrics Drawer	Admin Only	Displays Stripe transaction summaries and plan lists.

6. COMPLETE CURRICULUM & SAMPLE PROGRAM LIBRARY

6.1 Standard Categories

LogicLens organizes beginner Java programs into 5 distinct categories, covering typical introductory CS curriculum structures:

- 1. Basic Math & Input/Output: Celsius/Fahrenheit conversions, temp swaps, simple calculator structures.
- 2. Conditionals: Greatest of three numbers, leap year checks, digit vs special char validations.
- 3. Loop Structures: Factorials, sum of natural numbers, multiplication tables.
- 4. Number Logic Programs: Palindromes, Prime validations, Armstrong calculations, Fibonacci sequence.
- 5. Pattern Programs: Inverted pyramids, diamond patterns, Floyd's triangle, Pascal's triangle.

6.2 Mappings for Key Algorithms

The following table outlines how standard algorithms map to visual nodes and variables:

Algorithm Name	Key Variables Tracked	Decision Nodes Created	Loop Nodes Created
Prime Number Check	num, temp, i, isPrime	num <= 1, num % i == 0	i * i <= num
Palindrome Number	n, original, reversed, rem	original == reversed	n > 0
Fibonacci Series	n, first, second, next, i	i < n	i = 2; i < n; i++
Pyramid Stars	rows, i, space, j	space <= rows - i, j <= 2 * i - 1	i <= rows (Outer), Space & Star (Inner)

Demo-Critical Mappings

These programs are thoroughly configured with pre-analyzed variables and explanation tips to showcase the full features of the visualization engine.

7. PRODUCTION DEPLOYMENT & SPLIT CONFIG BLUEPRINT

7.1 Split-Platform Infrastructure Topology

LogicLens is optimized for a split-platform deployment architecture, dividing frontend delivery from server processing:

- Frontend Client Host: Vercel Static Web Hosting.
- API Server Host: Render Web Services.
- SQL Database Host: Neon Serverless PostgreSQL.

7.2 Critical Environment Configuration

Ensure the following environment variables are set in production:

```
# =====
# BACKEND API SERVER ENVIRONMENT (.env on Render)
# =====
NODE_ENV=production
PORT=5000
DATABASE_URL=postgresql://neondb_owner:***@ep-mute-wave.aws.neon.tech/logiclens
JWT_SECRET=super_secure_logiclens_infinite_token_system
FRONTEND_URL=https://logiclens-app.vercel.app
STRIPE_SECRET_KEY=sk_live_51M***
STRIPE_WEBHOOK_SECRET=whsec_***
GOOGLE_CLIENT_ID=***.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=***

# =====
# FRONTEND CLIENT ENVIRONMENT (Vercel Variables)
# =====
VITE_API_URL=https://logiclens-api.onrender.com
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_***
```

7.3 CORS & Authentication Policies

To allow split authentication, the backend CORS setup must permit request origins matching the frontend deployment url:

CORS Credential Policies

To support cookie-based sessions, CORS headers must set "credentials: true", and headers like "Access-Control-Allow-Origin" must specify the exact frontend url rather than the "*" wildcard.

8. FUTURE DEVELOPMENT ROADMAP & PLAYBOOK

8.1 Developer Handover: Step-by-Step Guides

1. How to Add a Sample Program to the Catalog

Create the program record inside the catalog library:

```
// Add program block inside database/src/seeds.ts
await db.insert(sampleProgramsTable).values({
  name: "Automorphic Number Check",
  category: "Number Logic Programs",
  subtype: "automorphic",
  code: "int num = 25;\nint sq = num * num;\nboolean isAutomorphic = true;\n// check logic...",
  description: "Check if the square of a number ends with the number itself.",
  difficulty: "intermediate",
  tags: ["math", "number-logic", "automorphic"]
});
```

2. How to Extend the Tracing Parser

To parse a new statement type (e.g., Array allocation `int[] arr = new int[5];`):

- Step A: Define the Statement type in `simulationEngine.ts` inside the `Statement` union.
- Step B: Add token matching rules to parse array declarations inside `parseStatement`.
- Step C: Update `execStatement` to support local array memory layouts in `VariableState`.

8.2 Future Enhancement Roadmaps

Phase 3: Time-Travel & Reversible Debugging

Implement a history queue: store the full `VariableState` diffs on each step. This allows developers to scrub backwards and restore memory variables instantaneously without reprocessing the simulation.

Phase 4: Power User Utilities

Implement SVG and PNG export tools for the flowchart canvas. Create a speed control slider to adjust auto-play intervals from 0.5x to 4x.

Phase 5: AI Insights & Smart Tutoring

Integrate the OpenAI/Anthropic SDKs into `/api/explain` to provide context-aware logic explanations and smart error detection for runtime failures.

Phase 6: Object-Oriented Visualizations

Expand JVM memory tracking to visualize instantiated objects and reference pointers in the Heap panel, linked to Stack references.